

## ESTRUCTURAS DE DATOS Y ALGORITMOS

JULIO 2013

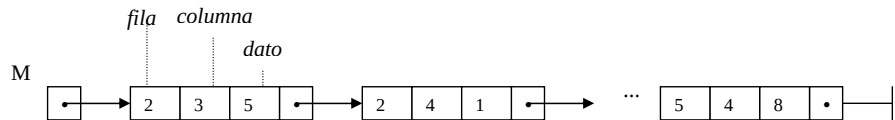
### 1. Matriz dispersa (1,5 puntos)

Una matriz dispersa es aquella que contiene en la mayoría de sus posiciones el valor cero. Por ejemplo:

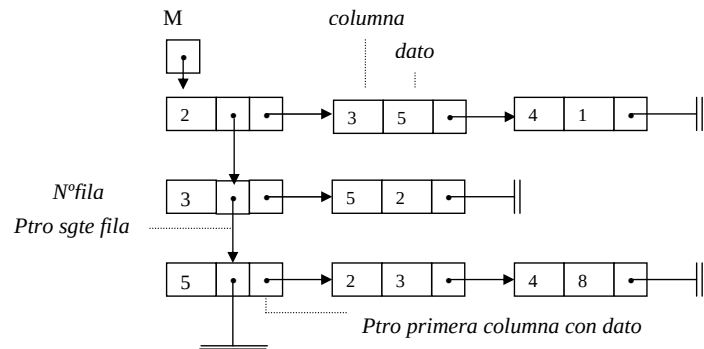
$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 5 & 1 & 0 \\ 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 8 & 0 \end{pmatrix}$$

Una matriz de este tipo se puede representar de diferentes maneras:

- **Representación A**, usando una lista simple con los elementos distintos de cero, ordenada ascendentemente por fila y columna:



- **Representación B**, una lista de las filas no vacías, ordenada por fila. Cada fila contiene una lista de las columnas distintas de cero con el número de columna y el dato y ordenada por número de columna.



Se pide:

- Escribir las declaraciones de clases de las representaciones A y B.
- Implementar el siguiente subprograma:

**public MatrizB convertir(MatrizA m)**

que genera la matriz resultado a partir de la matriz m, es decir, convierte de la representación A a la representación B.

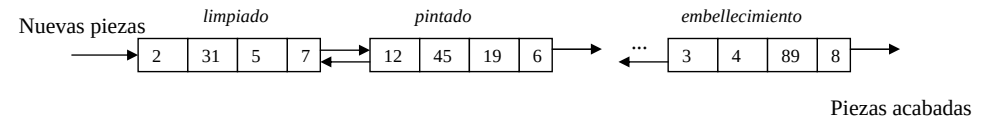
El algoritmo resultante deberá ser de coste  $O(N)$ , siendo N el número de elementos de la matriz m distintos de cero.

### 2. Fábrica (1,5 puntos)

Se quiere implantar un sistema de control en una fábrica de piezas de coches. La fábrica tiene 30 centros de procesado, y las piezas son procesadas de la siguiente manera:

- Cuando se empieza a procesar una pieza nueva, esta siempre entra a la planta de limpiado.
- A partir de ese momento, las piezas van moviéndose de una planta a otra, pero por motivos de organización, las piezas solo pueden pasar de una planta a otra consecutiva.
- Las piezas pueden moverse de una planta a otra en las dos direcciones, es decir, dadas 2 plantas de proceso consecutivas A y B, las piezas pueden pasar de A a B o de B a A.
- Las piezas finales saldrán de la planta de embellecimiento.

La siguiente figura muestra un ejemplo:



Queremos implementar el siguiente método:

```
public class Movimiento {
    Integer planta; // valores posibles: "entrada", "limpiado", "pintado",
                  // "planchado", ... "embellecimiento", "salida"
                  // codificados con enteros (-1 -> "entrada",
                  // 1 -> "limpiado", 30 -> "embellecimiento", 31 -> "salida")

    Integer num;
    // si planta = "entrada" indica un número de pieza que entra (a
    // la planta de limpiado)
    // si planta = "salida" indica que sale una pieza (de la
    // planta de embellecimiento). En este caso, el valor
    // de "num" no está definido
    // si no, "num" indica el número de piezas que se mueven de
    // una planta a otra
    String direccion; // valores posibles: "izq" o "der",
                    // indica el movimiento de las piezas
                    // este atributo solo tiene sentido si "planta"
                    // es distinto de "entrada" o "salida"
}

public class Fabrica {
    public SimpleLinkedList<Integer>
        procesar(SimpleLinkedList<Movimiento> lista)
    // Precondición: "lista" contiene los eventos producidos
    // Postcondición: la lista resultado contiene las piezas que han sido
    // acabadas (a partir de eventos de tipo "salida")
}
```

Por ejemplo, la lista inicial podría contener los siguientes eventos:

```
-1 47 // entra la pieza 47 (a la planta de limpiado)
-1 23 // entra la pieza 23 (a la planta de limpiado)
-1 35 // entra la pieza 35 (a la planta de limpiado)
1 2 der // 2 piezas pasan de la planta de limpiado (0) a la de su derecha
...
-1 147 // entra la pieza 147 (a la planta de limpiado)
31 // sale una pieza (de la planta de embellecimiento)
2 3 izq // 3 piezas pasan de la planta de pintado (1) a la de su izquierda
-1 88 // entra la pieza 88 (a la planta de limpiado)
31 // sale una pieza (de la planta de embellecimiento)
...
```

Se pide implementar el método “procesar”. Para ello se podrá hacer uso del TAD bicola.

**Bicola:**

```
public class Bicola<T> {

    public Bicola(); // constructora
    // Inicializa la bicola (vacía)

    public boolean estaVacía();
    // Indicará si la bicola está o no vacía.

    public void insertarIzq(T elemento);
    // añade el elemento E por el extremo izquierdo de la bicola

    public void insertarDer(T elemento);
    // añade el elemento E por el extremo derecho de la bicola

    public void eliminarIzq();
    // borra el elemento del extremo izquierdo de la bicola

    public void eliminarDer();
    // borra el elemento del extremo derecho de la bicola

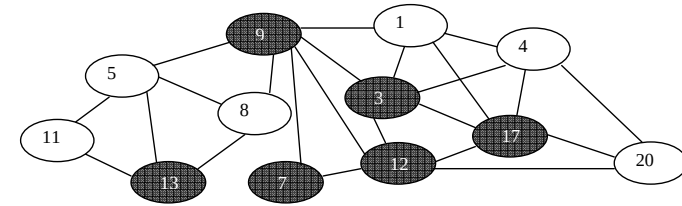
    public T obtenerIzq();
    // obtiene el elemento del extremo izquierdo de la bicola

    public T obtenerDer();
    // obtiene el elemento del extremo derecho de la bicola

}
```

### 3. Juego (1,5 puntos)

El gráfico siguiente representa un juego:



El objetivo del juego es llegar desde una casilla inicial a otra final. Para ello se debe respetar la regla fundamental del juego:

- Desde una casilla de un color solo se puede pasar a casillas de otro color (es decir, solo se puede pasar de blanco a negro o de negro a blanco).

Queremos hacer un algoritmo que, dadas dos casillas correspondientes al juego, nos devuelva un camino de longitud mínima que conecta las dos casillas. El algoritmo devolverá lo siguiente:

- La lista vacía, en caso de que las 2 casillas no puedan conectarse
- En caso de que haya varios caminos de longitud mínima se devolverá cualquiera de ellos

Por ejemplo, la llamada camino(“11”, “20”) devolverá la lista <11, 13, 8, 9, 1, 3, 4, 17, 20> (puede haber otros caminos de la misma longitud).

```
public class Casilla {
    String color; // "blanco" o "negro"
    int valor;
}

public class GraphAL<T> implements GraphADT<T>
{
    protected final int DEFAULT_CAPACITY = 100;
    protected int numVertices; // number of vertices in the graph
    protected boolean[][] adjMatrix; // adjacency matrix
    protected T[] vertices; // values of vertices
}

public class GrafoJuego extends GraphAL<Casilla> {

    public SimpleLinkedList<Casilla> camino(Casilla comienzo, Casilla final)

}
```

4. Evaluador (1,5 puntos)

Tenemos una lista que contiene una secuencia de instrucciones de la forma “x = y + z”, y una tabla hash que contiene las variables con sus valores. Se quiere hacer un programa que evalúe las instrucciones y vaya actualizando la tabla hash.

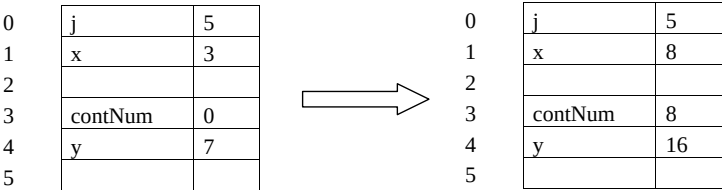
```
public class Instruccion {
    String nombre1;
    String nombre2;
    String nombre3;
    String operador; // un valor entre "+", "-", "/" y "*"
    // representa una instrucción de la forma:
    //                               nombre1 = nombre2 operador nombre3
}

public class TablaHashVariables extends HashMap<String, Integer> {
    public void evaluar(SimpleLinkedList<Instruccion>)
}
```

Por ejemplo, dada la siguiente lista de instrucciones:

listaInstrucciones: (x = x + j; contNum = contNum + x; y = contNum + x)

La tabla hash pasaría a ser modificada de la manera siguiente:



Se pide:

- Implementar el método evaluar
- Calcular su coste de manera razonada

Lista

```
public interface SimpleLinkedListADT<T> {

    public T removeFirst()
    public T removeLast()
    public T remove(T elem)
    public T first()
    public T last()
    public boolean contains(T elem)
    public boolean isEmpty()
    public int size()
    public Iterator<T> iterator()
}
```

Pila

```
public interface StackADT<T> {
    public void push(T elem)
    public T pop()
    public T peek()
    public boolean isEmpty()
    public int size()
}
```

Ilara / Cola

```
public interface QueueADT<T> {
    public void insert(T elem)
    public T remove()
    public T first()
    public boolean isEmpty()
    public int size()
}
```

Hash taula / Tabla hash

```
public interface Map<K, V> {
    public V put(K key, V value)
    public V get(K key)
    public T remove(K key)
    public boolean containsKey(K key)
    public int size()
}
```